🛡️ SupportPearlz AI: Comprehensive Project Reference Notes

1. Project Overview & Objectives

- **Project Name:** SupportPearlz AI – Enterprise Customer Support Assistant
- **Target Domain:** Pearlz Home Systems (Customer Support & Technical Documentation)
- **Core Purpose:** To provide a production-grade Retrieval-Augmented Generation (RAG) system that delivers accurate, strictly grounded answers to customer and support agent inquiries using official product manuals, warranties, and policy documents.
- **Key Design Philosophy:** Zero hallucination tolerance, robust security compliance, clean modular architecture, and zero-configuration cloud deployment capability.

2. Technology Stack

- **Language:** Python
- **Web Interface:** Streamlit (featuring custom CSS-styled professional UI, dual-screen layout, and chat history)
- **Orchestration & Framework:** LangChain (LCEL - LangChain Expression Language)
- **LLM Model:** OpenAI gpt-4o-mini (optimized for speed, cost efficiency, and structured reasoning)
- **Embeddings:** OpenAI text-embedding-3-small (for high-performance semantic representation)
- **Vector Store:** FAISS (Facebook AI Similarity Search) for lightweight, fast, local in-memory vector storage
- **Cloud Hosting:** Streamlit Cloud

3. Project File Structure & Architecture

The application is fully modularized into clean, single-responsibility packages:

Plaintext

supportpearlz/

├── data/

| ├── knowledge_base/ # 10 synthetic markdown files (.md)

| └── vector_store/ # Local persisted FAISS index files

```
├── src/
|   ├── chains/
|   |   └── rag_chain.py    # LCEL pipeline & LLM configuration
|   ├── ingestion/
|   |   └── build_index.py  # Document loader, chunker, & vector builder
|   ├── retrieval/
|   |   └── retriever.py    # FAISS loader & similarity search retriever
|   └── utils/
|       └── logging_setup.py# Centralized system logger
├── app.py                  # Main Streamlit application entry point
├── requirements.txt        # Pinned project dependencies
└── .gitignore              # Security control (ignores .env and vector store)
```

4. Pipeline Mechanics

A. Document Ingestion & Indexing Pipeline

1. **Source Loading:** DirectoryLoader reads all .md files from data/knowledge_base/ using TextLoader with UTF-8 encoding.
2. **Text Splitting:** Documents are broken down using RecursiveCharacterTextSplitter with a chunk size of 500 characters and an overlap of 50 characters to maintain policy clause integrity without diluting embedding quality.
3. **Embedding & Persistence:** Chunks are vectorized via OpenAI's text-embedding-3-small and persisted locally into data/vector_store/ using FAISS.

B. Query & Retrieval Pipeline

1. **Authentication Gate:** Users provide their OpenAI API key dynamically at runtime via a secure Streamlit login screen.
2. **Auto-Build Fallback:** On cloud boot, if the vector store index is missing, the app automatically executes build_vector_store() using session credentials.
3. **Semantic Similarity Search:** The user prompt queries the FAISS vector database to retrieve the top $k = 4$ most relevant text chunks.

4. **Grounded Generation:** Retrieved contexts are formatted and piped along with the user query into the gpt-4o-mini model through an explicit system prompt template enforcing strict contextual adherence.

5. Security & Rubric Compliance

- **Credential Protection:** No API keys or .env files are hardcoded or committed to GitHub, satisfying strict security requirements via .gitignore.
- **Runtime Authentication:** Credentials are securely handled via active Streamlit session state (st.session_state.api_key) and wiped upon session closure.
- **Guardrails & Hallucination Defense:** System prompts restrict the model exclusively to the provided context. Out-of-scope inquiries trigger explicit refusals directing users to standard escalation routes.

6. Viva Preparation: Key Technical Defenses

- **Why FAISS over a cloud database?** FAISS allows lightweight, local, low-latency similarity searches directly within the Python runtime environment without needing paid external cloud database subscriptions.
- **How is hallucination prevented?** Through a two-tier defense: semantic retrieval ($k = 4$ relevant chunks) combined with strict LCEL prompt constraints that forbid the LLM from fabricating outside knowledge.
- **How are cloud path discrepancies handled?** By using dynamic absolute path resolution (Path(__file__).resolve()) across all modules to ensure seamless compatibility regardless of the cloud server's working directory.